

LAB 3 — 数据库逻辑结构定稿

项目：复旦校园百事通问答系统
成员：俞楚凡、赵敬彦
日期：2026-04-29
目标 RDBMS：PostgreSQL 16（兼容性说明见各节末尾）

目录

- 1. 总体说明
- 2. 完整表结构说明
- 3. 主要约束设计说明
- 4. 业务规则的数据库层实现
- 5. 核心 SQL 草稿（DDL）
- 6. 索引设计
- 7. 代表性查询示例

1 总体说明

本文档在 Lab 2 关系模式初稿的基础上，对所有表的字段类型、约束（NOT NULL / UNIQUE / CHECK / DEFAULT / 外键级联策略）进行完整定稿，确保：

- 数据完整性：在数据库层强制枚举值、数值范围、格式规范；
- 引用完整性：通过外键级联策略（CASCADE / RESTRICT / SET NULL）保证关联数据一致；
- 业务规则落地：关键业务约束不依赖应用层，直接写入 DDL；
- 开发可用性：表结构、主键策略与索引设计足以支撑后续系统开发。

数据库层次结构（实体分组）

— 人员与权限模块	
— People	人员基表
— Student	学生扩展表
— Teacher	教师扩展表
— SysUser	系统用户表
— 空间地理模块	
— Campus	校区表
— Building	建筑表
— Location	地点表
— 组织、课程与活动模块	
— Department	院系表
— Course	课程表
— Teaching	教授关系表 (M:N)
— Enrollment	选课关系表 (M:N)
— Event	校园活动表
— EventParticipation	活动参与表 (M:N)

└─ 系统日志模块	
└─ QueryRecord	问答记录表

2 完整表结构说明

字段说明格式： 字段名 | 数据类型 | 约束 | 业务含义

2.1 People — 人员基表

系统全局人员索引，学生、教师均以此表为基础，采用**类表继承（Class Table Inheritance）**模式。

字段名	数据类型	约束	业务含义
people_id	SERIAL	PRIMARY KEY	人员唯一标识（自增）
name	VARCHAR(50)	NOT NULL	真实姓名
gender	CHAR(1)	NOT NULL, CHECK IN ('M','F','O')	性别：M男/F女/O其他
phone	VARCHAR(20)	UNIQUE	手机号（可空，唯一）
email	VARCHAR(100)	UNIQUE	电子邮箱（可空，唯一）

2.2 Student — 学生扩展表

继承 **People**，与 **People** 共享主键 **people_id**。

字段名	数据类型	约束	业务含义
people_id	INT	PRIMARY KEY, FK → People(people_id) ON DELETE CASCADE	学生人员 ID（共享 PK）
student_no	VARCHAR(20)	NOT NULL, UNIQUE	学号（校内唯一）
grade	VARCHAR(10)	NOT NULL, CHECK IN ('大一','大二','大三','大四','研一','研二','研三','博一','博二','博三','博四','其他')	年级
major	VARCHAR(100)	NOT NULL	专业名称
dep_id	INT	NOT NULL, FK → Department(dep_id) ON DELETE RESTRICT	所属院系

2.3 Teacher — 教师扩展表

继承 **People**，与 **People** 共享主键 **people_id**。

字段名	数据类型	约束	业务含义
people_id	INT	PRIMARY KEY, FK → People(people_id) ON DELETE CASCADE	教师人员 ID（共享 PK）
staff_no	VARCHAR(20)	NOT NULL, UNIQUE	工号（校内唯一）
title	VARCHAR(20)	NOT NULL, CHECK IN ('助教','讲师','副教授','教授','研究员','特聘教授','其他')	职称
dept_id	INT	NOT NULL, FK → Department(dep_id) ON DELETE RESTRICT	所属院系

2.4 SysUser — 系统用户表

原名 **User**，因 **USER** 是 SQL 保留字，改名为 **SysUser**。

字段名	数据类型	约束	业务含义
user_id	SERIAL	PRIMARY KEY	用户唯一标识（自增）
people_id	INT	NOT NULL, UNIQUE, FK → People(people_id) ON DELETE CASCADE	关联人员（一人一账号）
username	VARCHAR(50)	NOT NULL, UNIQUE	登录用户名（唯一）
password_hash	VARCHAR(255)	NOT NULL	bcrypt 哈希密码
role_type	VARCHAR(10)	NOT NULL, CHECK IN ('student','teacher','admin')	角色：学生/教师/管理员
verification_status	VARCHAR(10)	NOT NULL, DEFAULT 'pending', CHECK IN ('pending','verified','rejected')	实名认证状态，默认待审核
dep_id	INT	FK → Department(dep_id) ON DELETE SET NULL	所属院系（管理员可为空）
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	账号注册时间

2.5 Campus — 校区表

字段名	数据类型	约束	业务含义
campus_id	SERIAL	PRIMARY KEY	校区唯一标识
campus_name	VARCHAR(50)	NOT NULL, UNIQUE	校区名称（唯一），如邯郸、江湾、枫林、张江
address	VARCHAR(200)	NOT NULL	校区地址

2.6 Building — 建筑表

字段名	数据类型	约束	业务含义
building_id	SERIAL	PRIMARY KEY	建筑唯一标识
building_name	VARCHAR(100)	NOT NULL	建筑名称
campus_id	INT	NOT NULL, FK → Campus(campus_id) ON DELETE RESTRICT	所属校区
building_type	VARCHAR(20)	NOT NULL, CHECK IN ('教学楼', '宿舍楼', '食堂楼', '图书馆', '行政楼', '实验楼', '体育设施', '医疗卫生', '其他')	建筑类型（枚举）
description	TEXT	—	建筑描述（可空）
(UNIQUE)	—	UNIQUE(building_name, campus_id)	同一校区内建筑名唯一

2.7 Location — 地点表

字段名	数据类型	约束	业务含义
location_id	SERIAL	PRIMARY KEY	地点唯一标识
location_name	VARCHAR(100)	NOT NULL	地点名称
building_id	INT	NOT NULL, FK → Building(building_id) ON DELETE RESTRICT	所属建筑

字段名	数据类型	约束	业务含义
facility_type	VARCHAR(20)	NOT NULL, CHECK IN ('教室', '食堂', '咖啡店', '自习室', '图书馆', '实验室', '运动场地', '办公室', '医务室', '其他')	设施类型 (枚举)
description	TEXT	—	位置描述 (楼层、 门牌等)
open_time	VARCHAR(100)	—	开放时间 描述 (自 然语言)
(UNIQUE)	—	UNIQUE(location_name, building_id)	同一建筑 内地点名 唯一

2.8 Department — 院系表

注: manager_id 与 People 存在可推迟外键，以打破初始化时的循环依赖。

字段名	数据类型	约束	业务含义
dep_id	SERIAL	PRIMARY KEY	院系唯一标识
dep_name	VARCHAR(100)	NOT NULL, UNIQUE	院系名称 (全校 唯一)
contact_info	VARCHAR(200)	—	对外联系方式
office_location_id	INT	FK → Location(location_id) ON DELETE SET NULL	办公地点 (可 空)
manager_id	INT	FK → People(people_id) ON DELETE SET NULL DEFERRABLE	院系负责人 (可 空, 可推迟约 束)
description	TEXT	—	院系简介

2.9 Course — 课程表

字段名	数据类型	约束	业务含义
course_id	SERIAL	PRIMARY KEY	课程唯一标识
course_name	VARCHAR(200)	NOT NULL	课程名称
dep_id	INT	NOT NULL, FK → Department(dep_id) ON DELETE RESTRICT	开课院系
description	TEXT	—	课程简介

字段名	数据类型	约束	业务含义
(UNIQUE)	—	UNIQUE(course_name, dep_id)	同院系内课程名唯一

2.10 Teaching — 教授关系表 (M:N)

记录教师与课程在特定学期的教授关系。

字段名	数据类型	约束	业务含义
teacher_id	INT	NOT NULL, FK → Teacher(people_id) ON DELETE CASCADE	教师ID
course_id	INT	NOT NULL, FK → Course(course_id) ON DELETE CASCADE	课程ID
semester	CHAR(11)	NOT NULL, CHECK (semester ~ '^\\d{4}-\\d{4}-[12]\$')	学期，格式 2024-2025-1
(PK)	—	PRIMARY KEY (teacher_id, course_id, semester)	同一学期同一课程同一教师唯一

2.11 Enrollment — 选课关系表 (M:N)

记录学生选修课程的情况及成绩。

字段名	数据类型	约束	业务含义
student_id	INT	NOT NULL, FK → Student(people_id) ON DELETE CASCADE	学生ID
course_id	INT	NOT NULL, FK → Course(course_id) ON DELETE CASCADE	课程ID
semester	CHAR(11)	NOT NULL, CHECK (semester ~ '^\\d{4}-\\d{4}-[12]\$')	学期，格式 2024-2025-1
grade	NUMERIC(5,2)	CHECK (grade IS NULL OR (grade >= 0 AND grade <= 100))	成绩 0-100（可空，未录入时为NULL）
(PK)	—	PRIMARY KEY (student_id, course_id, semester)	同一学期同一课程同一学生唯一

2.12 Event — 校园活动表

字段名	数据类型	约束	业务含义
event_id	SERIAL	PRIMARY KEY	活动唯一标识

字段名	数据类型	约束	业务含义
event_name	VARCHAR(200)	NOT NULL	活动名称
event_type	VARCHAR(20)	NOT NULL, CHECK IN ('讲座','论坛','文艺演出','体育赛事','学术交流','招聘宣讲','志愿服务','其他')	活动类型 (枚举)
start_time	TIMESTAMP	NOT NULL	活动开始 时间
end_time	TIMESTAMP	CHECK (end_time IS NULL OR end_time > start_time)	活动结束 时间 (可 空)
location_id	INT	FK → Location(location_id) ON DELETE SET NULL	举办地点 (可空)
host_dep_id	INT	FK → Department(dep_id) ON DELETE SET NULL	主办院系 (可空)
description	TEXT	—	活动详情

2.13 EventParticipation — 活动参与表 (M:N)

字段名	数据类型	约束	业务含义
participant_id	INT	NOT NULL, FK → People(people_id) ON DELETE CASCADE	参与人员ID
event_id	INT	NOT NULL, FK → Event(event_id) ON DELETE CASCADE	活动ID
register_time	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	报名时间 (默认当前时间)
(PK)	—	PRIMARY KEY (participant_id, event_id)	每人每活动仅能参与一次

2.14 QueryRecord — 问答记录表

字段名	数据类型	约束	业务含义
record_id	SERIAL	PRIMARY KEY	记录唯一标识
user_id	INT	NOT NULL, FK → SysUser(user_id) ON DELETE CASCADE	发起查询的用户
raw_question	TEXT	NOT NULL	用户原始问题
query_time	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	查询时间

字段名	数据类型	约束	业务含义
query_result	TEXT	—	查询结果（可空，异步写入）

3 主要约束设计说明

3.1 主键（PRIMARY KEY）设计

表	主键策略	说明
People	SERIAL 自增整数	全局人员统一编号
Student/Teacher	复用 people_id（共享PK）	类表继承，避免主键冗余
SysUser	SERIAL 自增整数	独立账号ID，与 people_id 解耦
Campus/Building/Location	SERIAL 自增	地理实体独立编号
Department	SERIAL 自增整数	院系独立编号
Course/Event	SERIAL 自增整数	课程/活动独立编号
Teaching	(teacher_id, course_id, semester) 联合PK	同一教师同一课程同一学期唯一
Enrollment	(student_id, course_id, semester) 联合PK	同一学生同一课程同一学期唯一
EventParticipation	(participant_id, event_id) 联合PK	每人每活动只能参加一次
QueryRecord	SERIAL 自增整数	记录追加写入

3.2 外键（FOREIGN KEY）与级联策略

外键关系	级联策略	设计理由
Student.people_id → People	CASCADE	删除人员时同步删除学生档案
Teacher.people_id → People	CASCADE	同上
SysUser.people_id → People	CASCADE	删除人员时注销账号
Building.campus_id → Campus	RESTRICT	有建筑的校区不允许直接删除
Location.building_id → Building	RESTRICT	有地点的建筑不允许直接删除
Student.dep_id → Department	RESTRICT	有学生的院系不允许删除
Teacher.dept_id → Department	RESTRICT	有教师的院系不允许删除
Course.dep_id → Department	RESTRICT	有课程的院系不允许删除

外键关系	级联策略	设计理由
Department.manager_id → People	SET NULL	负责人离校后院系仍存在，负责人置空
Department.office_location_id → Location	SET NULL	地点变动时院系仍存在，办公地点置空
Event.location_id → Location	SET NULL	场地变更不影响活动记录
Event.host_dep_id → Department	SET NULL	主办院系撤销时活动记录保留
Teaching/Enrollment → Teacher/Student/Course	CASCADE	教师/学生/课程删除时联动清除关联
EventParticipation → People/Event	CASCADE	人员或活动删除时联动清除参与记录
QueryRecord.user_id → SysUser	CASCADE	用户注销时删除其查询历史

3.3 NOT NULL 约束

所有**核心业务字段**均设置 **NOT NULL**，具体包括：

- 所有实体的**名称类字段**（如 name、dep_name、campus_name 等）
- 所有**枚举类型字段**（如 gender、role_type、building_type 等）
- 所有**时间戳字段**（如 start_time、query_time 等）
- 所有**强制关联外键**（如 Building.campus_id、Student.dep_id 等）

允许为空的字段：可选信息（description、open_time、phone、email）、结束时间（end_time）、成绩（grade，未录入时为空白）、查询结果（query_result，异步写入）。

3.4 UNIQUE 约束

约束列	所在表	业务规则
phone	People	手机号全局唯一
email	People	邮箱全局唯一
student_no	Student	学号全校唯一
staff_no	Teacher	工号全校唯一
people_id	SysUser	一人只能注册一个账号
username	SysUser	登录用户名全局唯一
campus_name	Campus	校区名称唯一
(building_name, campus_id)	Building	同一校区内建筑名唯一
(location_name, building_id)	Location	同一建筑内地点名唯一
dep_name	Department	院系名称全校唯一

约束列	所在表	业务规则
(course_name, dep_id)	Course	同院系内课程名唯一

3.5 CHECK 约束（枚举与范围）

字段	表	允许值 / 规则
gender	People	'M'（男）/ 'F'（女）/ 'O'（其他）
grade	Student	'大一'~'大四'、'研一'~'研三'、'博一'~'博四'、'其他'
title	Teacher	'助教'/'讲师'/'副教授'/'教授'/'研究员'/'特聘教授'/'其他'
role_type	SysUser	'student' / 'teacher' / 'admin'
verification_status	SysUser	'pending' / 'verified' / 'rejected'
building_type	Building	'教学楼'/'宿舍楼'/'食堂楼'/'图书馆'/'行政楼'/'实验楼'/'体育设施'/'医疗卫生'/'其他'
facility_type	Location	'教室'/'食堂'/'咖啡店'/'自习室'/'图书馆'/'实验室'/'运动场地'/'办公室'/'医务室'/'其他'
event_type	Event	'讲座'/'论坛'/'文艺演出'/'体育赛事'/'学术交流'/'招聘宣讲'/'志愿服务'/'其他'
end_time > start_time	Event	结束时间必须晚于开始时间（允许空）
grade BETWEEN 0 AND 100	Enrollment	成绩范围 0~100，未录入时允许 NULL
semester ~ '^\\d{4}-\\d{4}-[12]\$',	Teaching/Enrollment	学期格式：2024-2025-1 或 2024-2025-2

3.6 DEFAULT 约束

字段	表	默认值	说明
verification_status	SysUser	'pending'	新账号默认待审核
created_at	SysUser	CURRENT_TIMESTAMP	自动记录注册时间
register_time	EventParticipation	CURRENT_TIMESTAMP	自动记录报名时间
query_time	QueryRecord	CURRENT_TIMESTAMP	自动记录查询时间

4 业务规则的数据库层实现

业务规则	实现方式
同一 <code>people_id</code> 不能同时出现在 Student 和 Teacher 中	应用层控制 + 可选触发器（见下方）
学号 / 工号全校唯一	UNIQUE 约束
一人只能有一个登录账号	<code>SysUser.people_id</code> UNIQUE
成绩合法性	CHECK (grade IS NULL OR (grade >= 0 AND grade <= 100))
活动结束时间不早于开始时间	CHECK (end_time IS NULL OR end_time > start_time)
学期格式标准化	CHECK (semester ~ '^\\d{4}-\\d{4}-[12]\$')
枚举字段只能取预定义值	CHECK IN (...) 约束
删除校区/建筑/院系前须清除下级数据	ON DELETE RESTRICT 外键
负责人/办公地点变动不影响院系记录	ON DELETE SET NULL 外键
查询记录只增不改（审计用）	应用层只执行 INSERT，数据库层可用视图屏蔽 UPDATE

触发器建议（防止同一人同时是学生和教师）

```
-- PostgreSQL 触发器示例
CREATE OR REPLACE FUNCTION check_person_role()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_TABLE_NAME = 'student' THEN
        IF EXISTS (SELECT 1 FROM Teacher WHERE people_id = NEW.people_id)
        THEN
            RAISE EXCEPTION '该人员已是教师，不能再注册为学生 (people_id=%) ',
NEW.people_id;
        END IF;
    ELSIF TG_TABLE_NAME = 'teacher' THEN
        IF EXISTS (SELECT 1 FROM Student WHERE people_id = NEW.people_id)
        THEN
            RAISE EXCEPTION '该人员已是学生，不能再注册为教师 (people_id=%) ',
NEW.people_id;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_student_role_check
BEFORE INSERT ON Student
FOR EACH ROW EXECUTE FUNCTION check_person_role();

CREATE TRIGGER trg_teacher_role_check
```

```
BEFORE INSERT ON Teacher
FOR EACH ROW EXECUTE FUNCTION check_person_role();
```

5 核心 SQL 草稿 (DDL)

目标方言: **PostgreSQL 16**

MySQL 8.0 兼容说明: 将 **SERIAL** 改为 **INT AUTO_INCREMENT**, 正则 **CHECK** 约束改为 **ENUM** 类型, **DEFERRABLE** 改为应用层控制。

```
-- =====
-- 复旦校园百事通问答系统 数据库建表脚本
-- 目标RDBMS: PostgreSQL 16
-- =====

-- 若需重建, 按依赖顺序删除
-- DROP TABLE IF EXISTS QueryRecord, EventParticipation, Enrollment,
Teaching,
-- Event, Course, SysUser, Teacher, Student, Department, Location,
Building, Campus, People CASCADE;

-- -----
-- 模块1: 空间地理 (无外部依赖, 最先建)
-- -----

CREATE TABLE Campus (
    campus_id SERIAL PRIMARY KEY,
    campus_name VARCHAR(50) NOT NULL UNIQUE,
    address VARCHAR(200) NOT NULL
);

CREATE TABLE Building (
    building_id SERIAL PRIMARY KEY,
    building_name VARCHAR(100) NOT NULL,
    campus_id INT NOT NULL
        REFERENCES Campus(campus_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    building_type VARCHAR(20) NOT NULL
        CHECK (building_type IN (
            '教学楼', '宿舍楼', '食堂楼', '图书馆',
            '行政楼', '实验楼', '体育设施', '医疗卫生', '其他'
        )),
    description TEXT,
    UNIQUE (building_name, campus_id)
);

CREATE TABLE Location (
    location_id SERIAL PRIMARY KEY,
    location_name VARCHAR(100) NOT NULL,
    building_id INT NOT NULL
        REFERENCES Building(building_id) ON DELETE RESTRICT ON UPDATE
CASCADE,
```

```

        facility_type    VARCHAR(20)        NOT NULL
        CHECK (facility_type IN (
            '教室', '食堂', '咖啡店', '自习室', '图书馆',
            '实验室', '运动场地', '办公室', '医务室', '其他'
        )),
        description       TEXT,
        open_time         VARCHAR(100),
        UNIQUE (location_name, building_id)
    );

-- -----
-- 模块2: 人员基表 (早于院系建立)
-- -----

CREATE TABLE People (
    people_id    SERIAL                PRIMARY KEY,
    name         VARCHAR(50)           NOT NULL,
    gender       CHAR(1)               NOT NULL
        CHECK (gender IN ('M', 'F', 'O')),
    phone        VARCHAR(20)           UNIQUE,
    email        VARCHAR(100)          UNIQUE
);

-- -----
-- 模块3: 院系 (依赖 Location、People, manager_id 用 DEFERRABLE)
-- -----

CREATE TABLE Department (
    dep_id       SERIAL                PRIMARY KEY,
    dep_name     VARCHAR(100)          NOT NULL UNIQUE,
    contact_info  VARCHAR(200),
    office_location_id INT
        REFERENCES Location(location_id) ON DELETE SET NULL ON UPDATE
CASCADE,
    manager_id   INT
        REFERENCES People(people_id) ON DELETE SET NULL ON UPDATE CASCADE
        DEFERRABLE INITIALLY DEFERRED,
    description  TEXT
);

-- -----
-- 模块4: 学生 / 教师扩展表 (依赖 People、Department)
-- -----

CREATE TABLE Student (
    people_id    INT                    PRIMARY KEY
        REFERENCES People(people_id) ON DELETE CASCADE ON UPDATE CASCADE,
    student_no   VARCHAR(20)           NOT NULL UNIQUE,
    grade        VARCHAR(10)           NOT NULL
        CHECK (grade IN (
            '大一', '大二', '大三', '大四',
            '研一', '研二', '研三',
            '博一', '博二', '博三', '博四', '其他'
        )),

```

```

    major          VARCHAR(100)    NOT NULL,
    dep_id          INT              NOT NULL
                REFERENCES Department(dep_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Teacher (
    people_id       INT              PRIMARY KEY
                REFERENCES People(people_id) ON DELETE CASCADE ON UPDATE CASCADE,
    staff_no        VARCHAR(20)      NOT NULL UNIQUE,
    title           VARCHAR(20)      NOT NULL
                CHECK (title IN (
                    '助教', '讲师', '副教授', '教授', '研究员', '特聘教授', '其他'
                )),
    dept_id         INT              NOT NULL
                REFERENCES Department(dep_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- -----
-- 模块5: 系统用户表 (依赖 People、Department)
-- -----

CREATE TABLE SysUser (
    user_id         SERIAL           PRIMARY KEY,
    people_id       INT              NOT NULL UNIQUE
                REFERENCES People(people_id) ON DELETE CASCADE ON UPDATE CASCADE,
    username        VARCHAR(50)      NOT NULL UNIQUE,
    password_hash   VARCHAR(255)     NOT NULL,
    role_type       VARCHAR(10)      NOT NULL
                CHECK (role_type IN ('student', 'teacher', 'admin')),
    verification_status VARCHAR(10)  NOT NULL DEFAULT 'pending'
                CHECK (verification_status IN ('pending', 'verified',
'rejected')),
    dep_id          INT
                REFERENCES Department(dep_id) ON DELETE SET NULL ON UPDATE
CASCADE,
    created_at      TIMESTAMP        NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- -----
-- 模块6: 课程 (依赖 Department)
-- -----

CREATE TABLE Course (
    course_id       SERIAL           PRIMARY KEY,
    course_name     VARCHAR(200)     NOT NULL,
    dep_id          INT              NOT NULL
                REFERENCES Department(dep_id) ON DELETE RESTRICT ON UPDATE
CASCADE,
    description     TEXT,
                UNIQUE (course_name, dep_id)
);

-- -----
-- 模块7: 教授关系 (M:N, 依赖 Teacher、Course)
-- -----

```

```
-----  
CREATE TABLE Teaching (  
    teacher_id INT NOT NULL  
        REFERENCES Teacher(people_id) ON DELETE CASCADE ON UPDATE CASCADE,  
    course_id INT NOT NULL  
        REFERENCES Course(course_id) ON DELETE CASCADE ON UPDATE CASCADE,  
    semester CHAR(11) NOT NULL  
        CHECK (semester ~ '^\\d{4}-\\d{4}-[12]$'),  
    PRIMARY KEY (teacher_id, course_id, semester)  
);
```

```
-----  
-- 模块8: 选课关系 (M:N, 依赖 Student、Course)  
-----
```

```
CREATE TABLE Enrollment (  
    student_id INT NOT NULL  
        REFERENCES Student(people_id) ON DELETE CASCADE ON UPDATE CASCADE,  
    course_id INT NOT NULL  
        REFERENCES Course(course_id) ON DELETE CASCADE ON UPDATE CASCADE,  
    semester CHAR(11) NOT NULL  
        CHECK (semester ~ '^\\d{4}-\\d{4}-[12]$'),  
    grade NUMERIC(5,2)  
        CHECK (grade IS NULL OR (grade >= 0 AND grade <= 100)),  
    PRIMARY KEY (student_id, course_id, semester)  
);
```

```
-----  
-- 模块9: 校园活动 (依赖 Location、Department)  
-----
```

```
CREATE TABLE Event (  
    event_id SERIAL PRIMARY KEY,  
    event_name VARCHAR(200) NOT NULL,  
    event_type VARCHAR(20) NOT NULL  
        CHECK (event_type IN (  
            '讲座', '论坛', '文艺演出', '体育赛事',  
            '学术交流', '招聘宣讲', '志愿服务', '其他'  
        )),  
    start_time TIMESTAMP NOT NULL,  
    end_time TIMESTAMP  
        CHECK (end_time IS NULL OR end_time > start_time),  
    location_id INT  
        REFERENCES Location(location_id) ON DELETE SET NULL ON UPDATE  
CASCADE,  
    host_dep_id INT  
        REFERENCES Department(dep_id) ON DELETE SET NULL ON UPDATE  
CASCADE,  
    description TEXT  
);
```

```
-----  
-- 模块10: 活动参与 (M:N, 依赖 People、Event)  
-----
```

```

-----
CREATE TABLE EventParticipation (
    participant_id INT NOT NULL
        REFERENCES People(people_id) ON DELETE CASCADE ON UPDATE CASCADE,
    event_id INT NOT NULL
        REFERENCES Event(event_id) ON DELETE CASCADE ON UPDATE CASCADE,
    register_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (participant_id, event_id)
);

```

```

-----
-- 模块11: 问答记录 (依赖 SysUser)
-----

```

```

CREATE TABLE QueryRecord (
    record_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL
        REFERENCES SysUser(user_id) ON DELETE CASCADE ON UPDATE CASCADE,
    raw_question TEXT NOT NULL,
    query_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    query_result TEXT
);

```

6 索引设计

在外键列和频繁查询列上建立索引，提升连接与过滤性能。

```

-- 地理模块：按校区查建筑、按建筑查地点
CREATE INDEX idx_building_campus ON Building(campus_id);
CREATE INDEX idx_location_building ON Location(building_id);
CREATE INDEX idx_location_type ON Location(facility_type);

-- 人员模块：按院系查学生/教师
CREATE INDEX idx_student_dep ON Student(dep_id);
CREATE INDEX idx_teacher_dept ON Teacher(dept_id);

-- 课程模块：按院系查课程、按学期查教授关系/选课
CREATE INDEX idx_course_dep ON Course(dep_id);
CREATE INDEX idx_teaching_semester ON Teaching(semester);
CREATE INDEX idx_enrollment_semester ON Enrollment(semester);
CREATE INDEX idx_enrollment_student ON Enrollment(student_id);

-- 活动模块：按时间、类型查活动
CREATE INDEX idx_event_start_time ON Event(start_time);
CREATE INDEX idx_event_type ON Event(event_type);
CREATE INDEX idx_event_host_dep ON Event(host_dep_id);

-- 日志模块：按用户查询记录、按时间排序

```



```
CREATE INDEX idx_queryrecord_user ON QueryRecord(user_id);
CREATE INDEX idx_queryrecord_time ON QueryRecord(query_time DESC);
```

7 代表性查询示例

7.1 查询某校区所有食堂

```
SELECT l.location_name, l.open_time, b.building_name
FROM Campus c
JOIN Building b ON b.campus_id = c.campus_id
JOIN Location l ON l.building_id = b.building_id
WHERE c.campus_name = '邯郸校区'
      AND l.facility_type = '食堂'
ORDER BY b.building_name;
```

7.2 查询某课程在某学期的授课教师

```
SELECT p.name, t.title, t.staff_no
FROM Teaching te
JOIN Teacher t ON t.people_id = te.teacher_id
JOIN People p ON p.people_id = t.people_id
JOIN Course c ON c.course_id = te.course_id
WHERE c.course_name = '数据库设计'
      AND te.semester = '2024-2025-2';
```

7.3 查询某学生的选课及成绩

```
SELECT c.course_name, d.dep_name, e.semester, e.grade
FROM Enrollment e
JOIN Student s ON s.people_id = e.student_id
JOIN Course c ON c.course_id = e.course_id
JOIN Department d ON d.dep_id = c.dep_id
WHERE s.student_no = '22300001'
ORDER BY e.semester, c.course_name;
```

7.4 查询近期校园讲座（未来30天内）

```
SELECT e.event_name, e.start_time, e.end_time,
       l.location_name, b.building_name, d.dep_name
FROM Event e
LEFT JOIN Location l ON l.location_id = e.location_id
LEFT JOIN Building b ON b.building_id = l.building_id
LEFT JOIN Department d ON d.dep_id = e.host_dep_id
```

```
WHERE e.event_type = '讲座'
      AND e.start_time BETWEEN NOW() AND NOW() + INTERVAL '30 days'
ORDER BY e.start_time;
```

7.5 统计各院系课程数量

```
SELECT d.dep_name, COUNT(c.course_id) AS course_count
FROM Department d
LEFT JOIN Course c ON c.dep_id = d.dep_id
GROUP BY d.dep_id, d.dep_name
ORDER BY course_count DESC;
```

7.6 查询用户最近10条问答记录

```
SELECT r.record_id, r.raw_question, r.query_time, r.query_result
FROM QueryRecord r
JOIN SysUser u ON u.user_id = r.user_id
WHERE u.username = 'zhangsan'
ORDER BY r.query_time DESC
LIMIT 10;
```

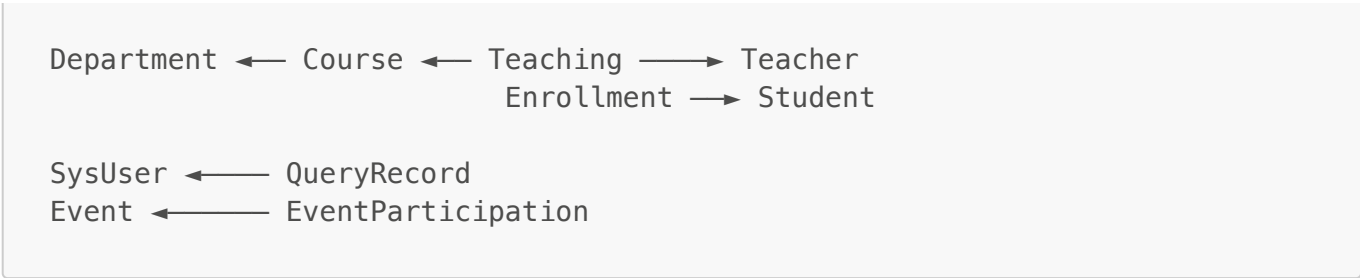
7.7 查询某院系举办过的所有活动及其参与人数

```
SELECT e.event_name, e.event_type, e.start_time,
       COUNT(ep.participant_id) AS participant_count
FROM Event e
JOIN Department d ON d.dep_id = e.host_dep_id
LEFT JOIN EventParticipation ep ON ep.event_id = e.event_id
WHERE d.dep_name = '计算机科学技术学院'
GROUP BY e.event_id, e.event_name, e.event_type, e.start_time
ORDER BY e.start_time DESC;
```

附录：外键依赖关系汇总

```
People ← Student → Department
People ← Teacher → Department
People ← SysUser → Department
People ← Department (manager_id, DEFERRABLE)
People ← EventParticipation → Event

Campus ← Building ← Location
Location ← Department (office_location_id)
Location ← Event
```



本文档为 Lab 3 定稿版，完整包含建表 DDL、约束说明及核心查询，可直接用于后续系统开发。